A Computational Companion for the Bevington Archetype: Solutions with `Fortran`, `Julia`, `Mathematica`, `Octave`, `Python`, and `Wolfram Alpha`.

Daniel Topa

January 13, 2023

Abstract

The linear least squares problem presented in Bevington's classic work has informed generations of scientists and engineers. As such, it has become an archetype problem in the broad field of linear least squares. Fitting data to a curve is a foundation process for interpreting experimental data, and fitting to a linear function is a first step. Bevington's presentation and discussion remain relevant and the goal here is to reproduce the results using modern vector computation tools

1 Problem Formulation

Bevington[1] begins with a sequence of $m = 9$ measurements $\{x_k, y_k\}_{k=1}^{m}$ encoding measurement position and temperature. Our challenge is to find the $n = 2$ parameters a for the linear function

$$y(x) = a_0 + a_1 x \tag{1}$$

which best describe the data in the l^2 norm. The fit parameter a_0 represents the intercept, and a_1 the slope. The associated error parameters σ_0 and σ_1 are an essential part of the solution. The least squares solution is defined as

$$x_{LS} = \underset{x \in \mathbb{C}^n}{\arg\min} \| \mathbf{A}\, a - y \|_2 \tag{2}$$

1.1 Mesh and Measurements

As we are using vector computing tools, we have the luxury of vector manipulation. Two vectors encode the measurements, a sequence of locations x, and temperature, y, at each location. A constant vector is added to augment the data set. For the Bevington data, these vectors have the

[1] The calculus-based solution [**?**, ch. 6] to the least squares problem is equivalent to the normal equations solution presented here.

following form:

$$\mathbf{1} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \quad x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \\ x_8 \\ x_9 \end{bmatrix}, \quad y = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \\ y_6 \\ y_7 \\ y_8 \\ y_9 \end{bmatrix}. \tag{3}$$

1.2 Linear System

The system matrix $\mathbf{A}$ encodes the mesh with the column composition $\mathbf{A} = \left[\begin{array}{cc} 1 & x \end{array} \right]$. This matrix maps the solution vector a to the measurement vector y as seen below.

$$\begin{array}{ccc} \mathbf{A} & a & = & y \\ \left[\begin{array}{cc} \mathbf{1} & x \end{array} \right] & \begin{bmatrix} a_0 \\ a_1 \end{bmatrix} & = & \left[\begin{array}{c} y \end{array} \right] \\ \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \\ 1 & 5 \\ 1 & 6 \\ 1 & 7 \\ 1 & 8 \\ 1 & 9 \end{bmatrix} & \begin{bmatrix} a_0 \\ a_1 \end{bmatrix} & = & \begin{bmatrix} 15.6 \\ 17.5 \\ 36.6 \\ 43.8 \\ 58.2 \\ 61.6 \\ 64.2 \\ 70.4 \\ 98.8 \end{bmatrix} \end{array} \tag{4}$$

2 Problem Solution

Because our problem has full column rank (that is the rank $\rho = n$), we can use the normal equations[2] to find the least squares solution.

3 Computational Tools

Let's briefly review the computational tools. The subset of open source tools is `Julia`, `Octave`, and, `Python`. While the `Wolfram Alpha` web site uses the proprietary `Mathematica` computation engine, it is freely available. However, it is limited to single commands, and not amenable to scripting. The proprietary `Mathematica` software is included in the discussion as it is the only tool to readily provide rational number computation.

[2] References abound. Some high-quality and readily accessible discussions are [**?**, §4.6], [**?**, §8.2], [**?**, §3.H], [**?**, §5.3.1].

3.1 Open Source Tools

`Python` is the brightest star in the constellation of open source tool, representing a de facto standard for computation.

`Octave` exploits the fact that much of the MATLAB application is based upon open source code. So much of the linear algebra routines used for this exercise are shared by the proprietary MATLAB and the open source `Octave`.

4 Computational Results

4.1 Input Mesh and Measurements

Input the system matrix by rows

`Julia`:
```
A = [ 1 1; 1 2; 1 3; 1 4; 1 5; 1 6; 1 7; 1 8; 1 9 ];
T = [15.6; 17.5; 36.6; 43.8; 58.2; 61.6; 64.2; 70.4; 98.8];
```

`Octave`:
```
A = [ 1 1; 1 2; 1 3; 1 4; 1 5; 1 6; 1 7; 1 8; 1 9 ]
T = [15.6; 17.5; 36.6; 43.8; 58.2; 61.6; 64.2; 70.4; 98.8]
```

`Python`:
```
import numpy as np
A = np.array([ [1, 1], [1, 2], [1, 3], [1, 4], [1, 5], [1, 6], [1, 7], [1, 8], [1,
9] ])
T = np.array([15.6, 17.5, 36.6, 43.8, 58.2, 61.6, 64.2, 70.4, 98.8])
```

`Mathematica`, `Wolfram Alpha`:
```
A = {{1, 1}, {1, 2}, {1, 3}, {1, 4}, {1, 5}, {1, 6}, {1, 7}, {1, 8}, {1, 9};
y = {15.6, 17.5, 36.6, 43.8, 58.2, 61.6, 64.2, 70.4, 98.8};
```

`Fortran`:
```
use, intrinsic :: iso_fortran_env, only : REAL64
integer, parameter :: m = 9, n = 2
integer, parameter :: rp = selected_real_kind ( REAL64 )
real ( kind = rp ), parameter :: A ( 1 : m , 1 : n ) = &
reshape ( [ [ 1.0_rp,1.0_rp,1.0_rp,1.0_rp,1.0_rp,1.0_rp,1.0_rp,1.0_rp,1.0_rp ],&
[ 1.0_rp,2.0_rp,3.0_rp,4.0_rp,5.0_rp,6.0_rp,7.0_rp,8.0_rp,9.0_rp ] ], [ m, n ])
real ( kind = rp ), parameter :: b ( 1 : m ) = &
[ 15.6_rp, 17.5_rp, 36.6_rp, 43.8_rp, 58.2_rp, 61.6_rp, 64.2_rp, 70.4_rp, 98.8_rp ]
```

4.2 Compute Least Squares Solution: Intrinsic Routines

All the software packages provide internal routines which can solve the least squares problem.

Julia:

```
julia> A  T
2-element Vector{Float64}:
4.813888888888871
9.408333333333335
```

Octave:

```
A = [ 1 1; 1 2; 1 3; 1 4; 1 5; 1 6; 1 7; 1 8; 1 9 ]
T = [15.6; 17.5; 36.6; 43.8; 58.2; 61.6; 64.2; 70.4; 98.8]
```

Python:

```
import numpy as np
A = np.array([ [1, 1], [1, 2], [1, 3], [1, 4], [1, 5], [1, 6], [1, 7], [1, 8], [1,
9] ])
T = np.array([15.6, 17.5, 36.6, 43.8, 58.2, 61.6, 64.2, 70.4, 98.8])
```

Mathematica, Wolfram Alpha:

```
A = {{1, 1}, {1, 2}, {1, 3}, {1, 4}, {1, 5}, {1, 6}, {1, 7}, {1, 8}, {1, 9};
y = {15.6, 17.5, 36.6, 43.8, 58.2, 61.6, 64.2, 70.4, 98.8};
```

Julia:

4.3 Compute Least Squares Error

References